

Exercise 5

Multi-File Programs

For this exercise, you're going to fill in the missing parts of a program that consists of multiple implementation and header files. This will require creating some global variables and functions and advertising them to other components via header files. You can download a starter for this program and supporting files from the course Moodle page, or, if you're logged in to remote.eos.ncsu.edu, you can use the following shell command to copy the needed files to the current directory:

```
cp '/mnt/coe/workspace/csc/CSC230/ex/exercise05/*' .
```

Rock-Paper-Scissors Tournament

The main program you're building is called **tournament**. Its job is to determine the outcome of a series of games of rock-paper-scissors and report a summary at the end. You are probably already familiar with rock-paper-scissors. For a game, each of the two players throws (makes a hand gesture for a shape) rock, paper or scissors. Paper beats rock, rock beats scissors and scissors beat paper. If the two players throw the same shape, it's a tie. If you're interested, you'll find a more detailed explanation at the game's Wikipedia page, https://en.wikipedia.org/wiki/Rock_paper_scissors

The program will read input from standard input, with each line of input representing a game. Every line contains two words, first the shape thrown by player A (**rock**, **paper** or **scissors**, all lower-case), then, after some whitespace, the shape thrown by player B. For each input line, the program should print one of the following lines to standard output, depending on who wins the game:

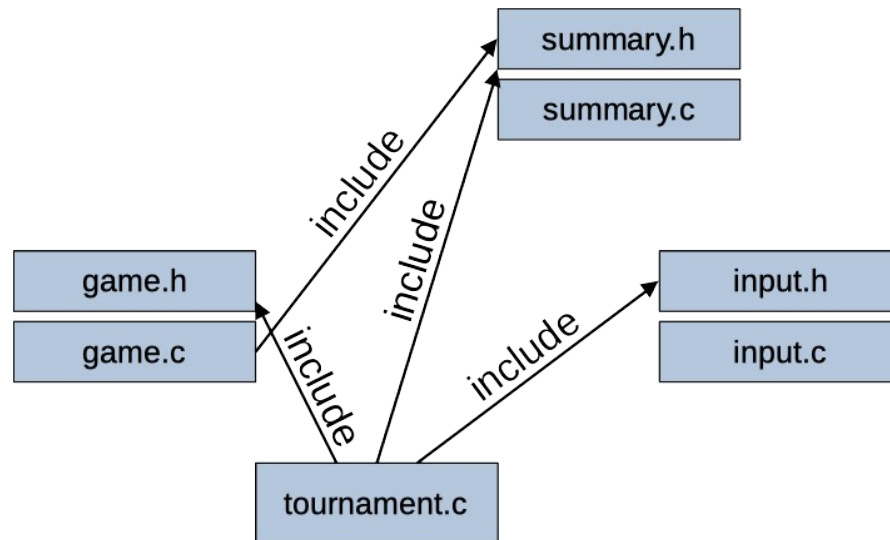
```
Player A wins
Player B wins
Players tie
```

Then, after printing the results of all the games, the program will print a summary table like the following. The starter code already contains a function to print this table for you.

```
+-----+-----+
| A Wins |      3 |
+-----+-----+
| B Wins |      1 |
+-----+-----+
| Ties   |      1 |
+-----+-----+
```

Program Organization

The source code for the program consists of four components. The `tournament.c` file is the top-level component. It contains the `main()` function, and, since it's not used by any of the other components, it doesn't need a header file.



The input component contains a function for reading words from the input. The summary component contains three global integers for counting the number of wins by player A, wins by player B and ties. It also contains a function for printing the summary table at the end. The game component contains a function for determining the winner of each game, printing the correct message and incrementing the appropriate counter from the summary component.

Since the `tournament.c` file uses functions provided by the other three components, it needs to include their header files. Also, since the `game.c` file uses global variables defined in `summary.c`, it will need to include the header file for the summary component.

Completing the Implementation

The starter contains files for each component, but you will need to fill in some missing code in each file. Here's what you'll need to do to get a working program:

- The `getInitial()` function in `input.c` needs to be implemented. It's job is to read the next word from standard input and return just the first letter of the word. In this function, you should skip any initial whitespace, then read non-whitespace characters up to the next whitespace or the end-of-file. Since we don't know how to use strings yet (soon), we'll just use the first letter of each word to tell whether the input said rock, paper or scissors. The function will need to read in all the characters of the next word, even though it's just returning the first one. You can use `getchar()` to read from standard input a character at a time. If there's no next word on standard input (i.e., you reach EOF), the function should return zero.
- The `decideGame()` function in `game.c` needs to be implemented. Based on the initial letter (**r**, **p** or **s**) for the shape thrown by each player, this function should print a

message to the terminal indicating which player won, and it should increment the appropriate counter in the summary component. If the input contains a word that doesn't start with **r**, **p** or **s**, the program should print a line saying "Invalid character" and exit with an exit status of 1. The starter code already has a function to help with this.

- The game, summary and input components each define a function that's used by `main()`. You need to add prototypes for each of these functions to the component's header. This will let `main()` know about these functions when it includes each component's header. The game component doesn't need a prototype for its `invalid()` function. This function is marked as static. It's an implementation detail of the game component and isn't intended to be used by the other components.
- The header for summary needs extern variable declarations for its three counters, `AWinTotal`, `BWinTotal` and `TieTotal`. The game component uses these counters, and the extern variable declarations will let it find out about them when it includes `summary.h`.
- The implementation files for game, summary and input each need to include their own header up at the top. This isn't absolutely necessary for the code to compile, but it's a good technique for making sure the header and the implementation file for a component are consistent.
- The `game.c` implementation file needs to include the header for summary, since it uses the three global variables defined in the summary component.
- The `tournament.c` implementation needs to include the headers for the other three components.

Building the Program

Since the program consists of four components, you'll need to compile and link them all together to build a working executable. The best way to do this is to compile them each separately, then link the resulting object files together. For now, we'll do this the easy way, with one `gcc` command. The following command should tell `gcc` to compile each of our source files using the usual flags, then link everything together into an executable named `tournament`.

```
gcc -Wall -std=c99 -g tournament.c game.c summary.c input.c -o tournament
```

If you've made a mistake in your implementation, the compiler will report an error here. Now that we are building a program from multiple components, there are some new types of errors you might see:

- If you get a message like "implicit declaration of function ...", this is a compile-time problem. It means you are trying to call a function that the compiler doesn't know about yet. This could be because you're trying to call a function in a different

component and you forgot to put a prototype for that function in the component's header or you forgot to include the component's header.

- If you get a message like "... undeclared", this is also a compile-time problem, consider if you're trying to access a global variable defined in another component. Maybe you forgot to put an extern declaration for that variable in the component's header or you forgot to include the component's header.
- If you get a message like "undefined reference to ...", this is a link-time problem. You could get this error if one of your components didn't actually define a variable or function that some other component was trying to use, or if you forgot to list one of your implementation files in the gcc command when you tried to build the program.

Sample Execution

Once your program is working, you should be able to run with redirected input from the sample inputs provided. For example, the following shows input being read from sample input 1 and the output going to the terminal.

```
$ ./tournament < input-1.txt
Player B wins
Player A wins
Players tie
Player A wins
Player A wins
+-----+-----+
| A Wins |      3 |
+-----+-----+
| B Wins |      1 |
+-----+-----+
| Ties   |      1 |
+-----+-----+
```

To make sure your output is exactly right, you can save it to a file and compare it with the expected output. The following shows running the program with input from sample input 2. Output is sent to a file named output.txt. Immediately after running the program, we check its exit status to make sure it ran successfully. Then, we use diff to make sure the program's output matches the expected output.

```
$ ./tournament < input-2.txt > output.txt
$ echo $?
0
$ diff output.txt expected-2.txt
```

Submitting Your Work

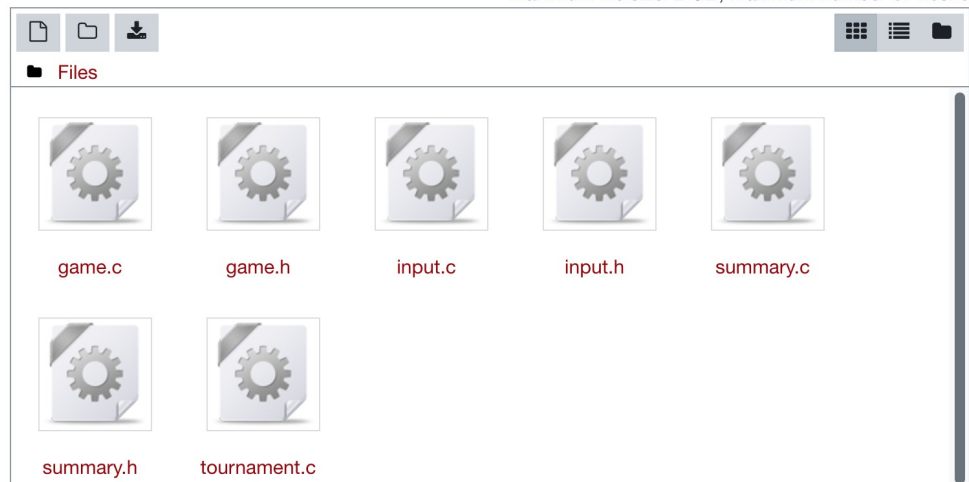
When you're done, submit all seven of your edited source files (four implementation files and three headers) to the exercise_05 assignment in Moodle. Be sure you submit your individual source files in the exercise_05 submission locker; don't put them in a subdirectory in the submission locker or in a zip file or other type of archive. This will prevent the grading program from finding your files and compiling them successfully.

There are a lot of files, but you should be able to add them all to your submission at once by dragging the files into the **Add Submission** box in Moodle. I tried submitting a solution, and here's what my submission window looked like before I clicked the **Save Changes** button.

▼ Add submission

File submissions

Maximum file size: 2 GB, maximum number of files: 8



Save changes

Cancel